

Respuestas a las Preguntas Teóricas — LAB 6

Pregunta 1

En el CMD del Dockerfile del backend se usa `--host 0.0.0.0`. Si en vez de eso arrancas con `--host 127.0.0.1` (o sin especificar host, que es el valor por defecto de Uvicorn), el contenedor arranca sin errores pero no es posible acceder a él desde el host. ¿Por qué? ¿Qué significa 0.0.0.0 en el contexto de red de un contenedor y por qué es necesario para que el mapeo de puertos funcione?

Respuesta:

La dirección 127.0.0.1 (localhost) hace referencia a la interfaz virtual de *loopback*, es decir, significa “escuchar solo las peticiones que se originan dentro de esta misma máquina”. Como un contenedor Docker actúa como una máquina aislada con su propia interfaz de red, usar 127.0.0.1 provoca que la API de Uvicorn solo admita peticiones enviadas desde **dentro del propio contenedor**, ignorando todo el tráfico exterior (incluyendo el de tu host).

Por otro lado, la dirección 0.0.0.0 es un comodín que significa “escuchar en **todas** las interfaces de red disponibles”. Al arrancar con `--host 0.0.0.0`, Uvicorn se pone a la escucha tanto en la red interna de loopback como en la red virtual (por ejemplo, `eth0`) que Docker le ha proporcionado. Cuando publicamos el puerto en Docker (`-p 8000:8000`), el motor de Docker reenvía el tráfico que le llega desde el sistema operativo anfitrión hacia la interfaz virtual externa del contenedor. Si la API no está escuchando en todas las interfaces mediante 0.0.0.0, el mapeo desde el host fracasará.

Pregunta 2

En el `docker-compose.yml`, el frontend recibe la variable de entorno `BACKEND_URL=http://backend:8000`. Si cambiaras esa URL a `http://localhost:8000`, el frontend arrancaría correctamente pero fallaría al intentar llamar al backend. Explica por qué localhost dentro de un contenedor no resuelve al backend. ¿Qué mecanismo usa Docker Compose para que backend sea un nombre DNS válido dentro de la red interna?

Respuesta:

Para un contenedor, localhost siempre se refiere a **sí mismo**. Si el contenedor del frontend intenta comunicarse con `http://localhost:8000`, estará buscando un servicio en el puerto 8000 **dentro de él mismo** y, como ahí solo corre la aplicación de Gradio (puerto 7860), la conexión será rechazada (*Connection Refused*). Para permitir la comunicación entre contenedores, Docker Compose crea automáticamente una **red puente privada** donde conecta todos los servicios definidos en el fichero. Además, implementa un **servidor DNS interno** que resuelve los nombres de los servicios. Esto significa que cada contenedor puede acceder a otro utilizando el nombre definido en `services:`. Así, el hostname `backend` se traduce internamente a la dirección IP real del contenedor correspondiente, permitiendo que el frontend se comunique correctamente con el backend.

Pregunta 3

En los Dockerfile de ambos servicios, las instrucciones siguen este orden:

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

¿Qué ocurriría con los tiempos de construcción si se invirtiera el orden y se hiciera `COPY .` antes del `RUN pip install`? Describe el mecanismo de caché de capas de Docker y en qué situaciones habituales del desarrollo (cambiar una línea en `main.py`, añadir una dependencia nueva) nota la diferencia.

Respuesta:

Si se invierte el orden, los tiempos de construcción aumentarían considerablemente ante cualquier cambio en el código.

Docker procesa cada instrucción del Dockerfile creando una **capa (layer)**. Si una capa no ha cambiado, Docker la reutiliza desde la **caché**. Sin embargo, si una capa cambia, todas las capas posteriores se invalidan automáticamente.

- **Orden actual (optimizado):** Si se modifica solo `main.py`, las capas de `COPY requirements.txt` y `RUN pip install` permanecen en caché. Solo se reconstruye la última capa (`COPY . .`), lo que es muy rápido.
- **Orden invertido (ineficiente):** Si primero se hace `COPY . .`, cualquier cambio en el código (aunque sea mínimo) invalida esa capa. Esto provoca que también se invalide la capa de `pip install`, obligando a reinstalar todas las dependencias en cada construcción.

Por tanto, el orden actual optimiza significativamente los tiempos de desarrollo, especialmente cuando se realizan cambios frecuentes en el código.

Pregunta 4

El fichero `.env` con el token de Hugging Face (`HF_TOKEN`) se referencia en el `docker-compose.yml` pero se añade al `.gitignore`. Existen al menos tres formas alternativas de pasar un secreto a un contenedor en producción: variable de entorno del shell, Docker Secrets (modo Swarm) y servicios externos de gestión de secretos (AWS Secrets Manager, HashiCorp Vault, etc.). Describe brevemente cada opción, identifica el riesgo principal de la solución con `.env` y explica en qué tipo de entorno de producción elegirías cada alternativa.

Respuesta:

Riesgo del uso de ficheros `.env`: Aunque esté en `.gitignore`, almacenar secretos en texto plano supone un riesgo, ya que pueden filtrarse accidentalmente (por copias, correos, errores humanos) o incluso quedar incluidos en una imagen Docker si no se gestiona correctamente.

1. **Variables de entorno del shell** *Descripción:* Se pasan dinámicamente en tiempo de ejecución sin almacenarse en disco. *Uso recomendado:* Entornos de desarrollo local y sistemas de integración continua (CI/CD).
2. **Docker Secrets / Kubernetes Secrets** *Descripción:* Los secretos se montan como archivos temporales en memoria (por ejemplo en `/run/secrets/`), evitando su persistencia en disco. *Uso recomendado:* Entornos orquestados (Docker Swarm o Kubernetes) con necesidades de control centralizado.
3. **External Secrets Managers (AWS Secrets Manager, Vault, etc.)** *Descripción:* Servicios externos que almacenan y gestionan secretos con cifrado, control de acceso y rotación automática. *Uso recomendado:* Entornos productivos complejos, especialmente en empresas con altos requisitos de seguridad y cumplimiento normativo.